PALL-FIN: Financial Data Aggregation & Analysis

1. Project Abstract

PALL-FIN is a high-performance financial data aggregation engine designed to analyze massive datasets of stock market transactions (Buy/Sell/Volume) using parallel computing techniques. The project serves as a practical implementation for a Design and Analysis of Algorithms (DAA) evaluation, demonstrating the shift from an O(N) sequential bottleneck to an O(N/K + K log K) parallel architecture using multithreading, custom hashing, and tournament reduction trees.

2. System Architecture

The project consists of two core decoupled systems:

- **C-based Backend Engine (app.c)**: Performs the heavy lifting. It parses multi-gigabyte CSV feeds, hashes ticker symbols, and merges the data utilizing POSIX threads (pthreads).
- **React/TypeScript Dashboard**: A visual telemetry interface that consumes the output of the backend to model time complexity, visualize thread allocations, and chart Amdahl's Speedup Efficiency.

3. Algorithm Breakdown

3.1 Standard Sequential Algorithm

The naive approach processes the CSV file line-by-line in a single thread.

- **Time Complexity**: **O(N)** where N is the number of rows.
- **Space Complexity**: **O(U)** where U is the number of unique stock tickers.
- **Bottleneck**: CPU clock speed and disk I/O wait times. A single core must wait for every character to be read sequentially.

3.2 Parallel Map-Reduce Algorithm

To bypass the sequential bottleneck, we designed a multithreaded architecture.

Phase 1: O(1) Boundary Resolution (Data Partitioning)

Instead of a master thread dealing out work, the file is mathematically divided by its byte size S across K threads. Each thread seeks to its byte offset (S/K) * i. To prevent tearing a line in half, the thread advances its pointer until it hits the next newline character \n.
This achieves O(1) partitioning with zero locking overhead.

Phase 2: O(N/K) Independent Parsing & Hashing

Each thread independently parses its chunk of N/K records. To aggregate data (Buy/Sell volumes) per ticker, we utilize the **djb2 hash function** to index into a local 64-slot open-addressed hash table.

- **djb2 Hash Function**: Known for excellent distribution with short strings (like stock tickers).
- **Local Tables**: By keeping hash tables local to the thread, we completely eliminate mutex lock contention during the parsing phase.

Phase 3: O(K log K) Tournament Reduction Tree Merge

Once all K threads finish parsing, we have K independent hash tables. To merge them:

1. Each thread runs a quicksort (qsort) on its local table: **O(U log U)**.
2. The threads merge their sorted arrays in a binary tournament tree structure. Pairs of arrays are merged linearly **O(U)**, halving the number of active arrays at each step.

- This creates a reduction tree of depth **log K**, making the merge phase highly efficient.

4. Dashboard Visualizations

The dashboard translates these theoretical concepts into empiric visualizations:

1. **Asymptotic Complexity Analysis**: A table comparing the mathematical proofs of our algorithms.
2. **Parallel Architecture Flow**: A visual map tracing data from the raw CSV, through the djb2 hash threads, down the reduction tree, to the aggregated state.
3. **Execution Time vs Input Size (N)**: A graph plotting execution time across varying dataset sizes. This visually proves that the parallel O(N/K) algorithm grows significantly slower than the sequential O(N) algorithm as the dataset scales up.
4. **Amdahl Speedup Efficiency**: Compares our measured multithreaded speedup against the theoretical limit defined by Amdahl's Law (speedup = 1 / ((1-p) + p/N)).

5. Conclusion

By applying DAA principles—specifically divide-and-conquer methodologies, efficient string hashing (djb2), and reduction trees—we successfully transformed an I/O and CPU bound financial parsing script into a scalable, high-throughput parallel engine.